

Outcome Prediction Quiz: Basics

95 points

Answer sheet. Mark every answer and write your reasoning on the separate answer sheet at the end of this booklet (pages S1–S2). Only the answer sheet is graded; the option lists in the questions are for reference.

Instructions. Predict the outcomes of the following experiments. You will have 40 minutes for 8 questions (5 min each). Each question changes the standard d8 setup from the assignment by applying its diff to the student repository (deep-learning-alchemy/assignments, branch a1_release, commit baafc3f); Appendix A gives the scoring code that maps the resulting runs to answers. Briefly justify each prediction by citing your assignment’s experiments. You will get partial credit (up to 75% of the value of the problem) if your rationale and citations are sound.

Grading and expectations. Outcome prediction is very difficult, even for experts. Do not panic if you do not have confidence in getting 90+%. Our expectation is that strong students will maybe be able to attain 80% and most scores will hover around 50%.

Due to the difficulty of this task, we will have two sources of partial credit. For many problems where exact numerical prediction is challenging, you will get up to 50% of the points if you’re only off by one cell. Each question’s scoring line states whether this applies and how many points it is worth. We will also provide partial credit for your rationale, in which you must cite your assignment for justification (see above).

Default setup. The standard setup of the Basics assignment: Llama-style architecture with 8 layers, hidden size 512, 8 attention heads, a 4k vocab size tokenizer, and about 35M parameters (d8); 614.4M training tokens of fresh data (no epoching); AdamW with $\beta_1, \beta_2 = (0.9, 0.95)$, linear decay learning rate schedule with 1% linear warmup, learning rate 0.003, context length 1024, batch size 64, weight decay 0.1 (masked), and 1.0 grad clipping.

Prediction

Q1. What normal training noise looks like

Difficulty: Easy

10 points

We train the standard recipe at ten paired seeds, setting $\text{model_seed} = \text{data_seed} = s$ for $s = 4, \dots, 13$, all on the same GPU type:

```
--- a/experiments/a1_basics/p3_measuring_variation.py
+++ b/experiments/a1_basics/p3_measuring_variation.py
@@ -3,14 +3,8 @@ from train import TrainConfig

RUNS = [
-    TrainConfig(
-        deterministic=True,
-        run_name_suffix="deterministic-reference-1",
-    ),
-    TrainConfig(
-        deterministic=True,
-        run_name_suffix="deterministic-reference-2",
-    ),
+    TrainConfig(model_seed=seed, data_seed=seed, run_name_suffix=f"quiz-s2-seed{seed}")
+    for seed in range(4, 14)
]
```

(a) The sample standard deviation $\sqrt{\sum_i (x_i - \bar{x})^2 / 9}$ of the ten final validation losses is closest to:

0.0003 0.001 0.003 0.01 0.03

(b) Define a run's *late-training jitter* as the standard deviation of (per-step train loss – its centered 51-step running mean), computed over the final 2000 optimizer steps. The average jitter of the ten runs is closest to:

0.004 0.013 0.04 0.13

Scoring: 5 points per part, for the listed value closest in ratio ($|\log(\text{measured}/\text{option})|$ smallest) to the key runs' value. Part (b) only: 2 points for an adjacent option.

Mark your answer and write your reasoning on answer sheet page S1.

Q2. Loss-curve microstructure

Difficulty: Easy

10 points

Using the late-training jitter defined in the first question (the standard deviation of per-step train loss minus its centered 51-step running mean, over the final 2000 optimizer steps), we rerun the standard recipe on the same 600,000 sequences at two other batch sizes, next to a standard batch-64 reference:

```
--- a/experiments/a1_basics/p5_loss_curve_augury.py
+++ b/experiments/a1_basics/p5_loss_curve_augury.py
@@ -3,7 +3,9 @@ from train import TrainConfig
```

```

RUNS = [
-   TrainConfig(),
+   TrainConfig(run_name_suffix="quiz-q1-standard-reference"),
+   TrainConfig(batch_size=16, run_name_suffix="quiz-q1-batch-size-16"),
+   TrainConfig(batch_size=256, run_name_suffix="quiz-q1-batch-size-256"),
]

```

For each batch size, check the option closest to the ratio of its jitter to the batch-64 reference run's jitter:

	$\times 1/4$	$\times 1/2$	$\times 1$	$\times 2$	$\times 4$		$\times 1/4$	$\times 1/2$	$\times 1$	$\times 2$	$\times 4$
batch size 16	.	.	.	.	.	batch size 256	.	.	.	.	.

Scoring: 5 points per run. The correct option is the listed ratio closest (in log ratio) to the key runs' measured jitter ratio. 2 points for answers that are off-by-one.

Mark your answer and write your reasoning on answer sheet page S1.

Q3. Skipping warmup

Difficulty: Easy

10 points

Starting from the standard recipe, we remove the learning-rate warmup entirely and keep everything else unchanged:

```

--- a/experiments/a1_basics/p1_hyperparameters.py
+++ b/experiments/a1_basics/p1_hyperparameters.py
@@ -3,7 +3,8 @@ from train import TrainConfig

```

```

RUNS = [
-   TrainConfig(),
+   TrainConfig(run_name_suffix="quiz-s1-standard"),
+   TrainConfig(warmup_percent=0.0, run_name_suffix="quiz-s1-no-warmup"),
]

```

Let $\Delta = L_{\text{no warmup}} - L_{\text{standard}}$ be the difference in final validation loss. Δ falls in:

- . $\Delta < 0.01$
- . $0.01 \leq \Delta < 0.04$
- . $0.04 \leq \Delta < 0.15$
- . $0.15 \leq \Delta < 0.5$
- . $\Delta \geq 0.5$, or the run diverges

Scoring: 10 points for the interval containing the key runs' Δ ; 2 points for an adjacent interval.

Mark your answer and write your reasoning on answer sheet page S1.

Q4. Trimming the decay phase

Difficulty: Medium

10 points

The wsdX schedule holds the peak learning rate constant after the standard 1% warmup, then decays linearly to zero over only the final fraction X of steps. We run five decay fractions and a default linear reference:

```

--- a/experiments/a1_basics/p1_hyperparameters.py
+++ b/experiments/a1_basics/p1_hyperparameters.py
@@ -3,7 +3,11 @@ from train import TrainConfig

```

```

RUNS = [
-   TrainConfig(),
+   TrainConfig(run_name_suffix="quiz-q2-linear-reference"),
+   *(
+       TrainConfig(lr_schedule=f"wsd{x}", run_name_suffix=f"quiz-q2-wsd{x}")
+       for x in [0.02, 0.05, 0.1, 0.2, 0.5]
+   ),
+ ]

```

What is the smallest X whose final validation loss comes within 0.02 of the default linear run's?

- $X = 2\%$
- $X = 5\%$
- $X = 10\%$
- $X = 20\%$
- $X = 50\%$
- none of these

Scoring: the correct option is the smallest listed X with $\text{loss}(\text{wsd}X) - \text{loss}(\text{linear}) < 0.02$, both losses from the key runs; 3 points for an adjacent option.

Mark your answer and write your reasoning on answer sheet page S1.

Q5. Peak learning rate and warmup

Difficulty: Medium

15 points

We change the peak learning rate and warmup fraction together:

```

--- a/experiments/a1_basics/p1_hyperparameters.py
+++ b/experiments/a1_basics/p1_hyperparameters.py
@@ -3,7 +3,12 @@ from train import TrainConfig

RUNS = [
-   TrainConfig(),
+   TrainConfig(
+       learning_rate=lr,
+       warmup_percent=wu,
+       run_name_suffix=f"quiz-q3-lr{lr}-warmup{wu}",
+   )
+   for lr, wu in [(0.009, 0.05), (0.027, 0.05), (0.027, 0.2)]
+ ]

```

For each configuration, check the bin containing its final validation loss:

lr	warmup	< 2.94	2.94–3.05	3.05–3.6	≥ 3.6 or NaN
0.009	5%	.	.	.	.
0.027	5%	.	.	.	.
0.027	20%	.	.	.	.

Scoring: 5 points per row; check the one bin containing the key run's final validation loss (bins are exhaustive; a boundary value belongs to the lower bin); 2 points per row for an adjacent bin.

Mark your answer and write your reasoning on answer sheet page S2.

Q6. Norm trajectories under schedule and weight decay

Difficulty: Medium

10 points

We train four runs that cross the learning-rate schedule (linear decay or constant) with the weight decay (0.1 or 0.0). Every step, each run logs the total parameter L2 norm and the total pre-clip gradient L2 norm. Predict how these two norms behave over the course of training.

```
--- a/experiments/al_basics/p6_activation_gradient_norms.py
+++ b/experiments/al_basics/p6_activation_gradient_norms.py
@@ -0,0 +1,36 @@
+from metric_logging import AFTER_BACKWARD, MetricLogger
+from modal_train import launch_training_jobs
+from train import TrainConfig
+
+
+def log_weight_and_grad_norms(ctx):
+    # Runs after backward and before gradient clipping, so grad_norm is pre-clip.
+    weight_sq = 0.0
+    grad_sq = 0.0
+    for parameter in ctx.model.parameters():
+        weight_sq = weight_sq + parameter.detach().float().pow(2).sum()
+        if parameter.grad is not None:
+            grad_sq = grad_sq + parameter.grad.detach().float().pow(2).sum()
+    return {"weight_norm": weight_sq*0.5, "grad_norm": grad_sq*0.5}
+
+
+RUNS = [
+    TrainConfig(
+        lr_schedule=schedule,
+        weight_decay=weight_decay,
+        metric_loggers=(
+            MetricLogger(event=AFTER_BACKWARD, fn=log_weight_and_grad_norms),
+        ),
+        run_name_suffix=f"quiz-q4-{schedule}-wd{weight_decay:g}",
+    )
+    for schedule in ["linear", "constant"]
+    for weight_decay in [0.1, 0.0]
+]
+
+
+def main():
+    launch_training_jobs(RUNS)
+
+
+if __name__ == "__main__":
+    main()
```

(a) Let $W(t)$ be the total parameter L2 norm (all weights) at step t . For each run, check where W at the end of training sits relative to its maximum over the whole run:

schedule	wd	at its peak (within 3%)	3–15% below peak	> 15% below peak
linear	0.1	.	.	.
linear	0.0	.	.	.
constant	0.1	.	.	.
constant	0.0	.	.	.

(b) Check every run in which the pre-clip gradient norm at least *doubles* over training (median of the final 10% of steps $\geq 2 \times$ median of the first 10%):

linear, wd 0.1 linear, wd 0.0 constant, wd 0.1 constant, wd 0.0

Scoring: (a) 1.5 points per row, the option containing $W(\text{end}) / \max_t W(t)$; (b) 1 point per run, checked iff the measured ratio is ≥ 2 .

Mark your answer and write your reasoning on answer sheet page S2.

Q7. Extrapolating your scaling ladders

Difficulty: Hard

15 points

In Problem 2(a) you trained d4–d9 ladders for four recipes: the baseline (lr 0.003, linear decay), constant lr, dropout 0.2, and lr 0.03. Part (a) takes all four to d16 (16 layers, hidden 1024, 251.7M parameters, the same 614.4M training tokens). Part (b) returns to Problem 2(c) with two ladder-bending interventions at d4 and d9:

```
--- a/experiments/al_basics/p2_scaling_law_reliability.py
+++ b/experiments/al_basics/p2_scaling_law_reliability.py
@@ -3,27 +3,31 @@ from model_config import depth_model_config
from train import TrainConfig

-RUNS = []
-seen_configs = set()
-for depth in range(4, 10):
-    for learning_rate, lr_schedule, dropout in [
-        (0.003, "linear", 0.0),
-        (0.003, "constant", 0.0),
-        (0.003, "linear", 0.2),
-        (0.03, "linear", 0.0),
-    ]:
-        config_key = (depth, learning_rate, lr_schedule, dropout)
-        if config_key in seen_configs:
-            continue
-        seen_configs.add(config_key)
-        RUNS.append(
-            TrainConfig(
-                model_config=depth_model_config(depth),
-                learning_rate=learning_rate,
-                lr_schedule=lr_schedule,
-                dropout=dropout,
-            )
-        )
-
+# Problem 2(a)'s four recipes at d16 (compiled, one micro-batch: needs an
+# 80 GB GPU such as Modal's default H100).
+P2A_RECIPES = {
+    "base": {},
+    "constant": {"lr_schedule": "constant"},
+    "dropout0.2": {"dropout": 0.2},
+    "lr0.03": {"learning_rate": 0.03},
+}
+
+# Two Problem 2(c)-style interventions at d4 and d9.
+P2C_INTERVENTIONS = {
+    "epochs8": {"num_train_sequences": 75_000, "num_epochs": 8.0},
+    "noqknorm-lr0.006": {"qk_norm": False, "learning_rate": 0.006},
+}
+
+RUNS = [
+    TrainConfig(model_name="d16", run_name_suffix=f"quiz-q5-d16-{name}", **changes)
+    for name, changes in P2A_RECIPES.items()
+] + [
```

```
+ TrainConfig(
+     model_config=depth_model_config(depth),
+     run_name_suffix=f"quiz-q5-{name}-d{depth}",
+     **changes,
+ )
+ for name, changes in P2C_INTERVENTIONS.items()
+     for depth in [4, 9]
+ ]
```

TODO: Add your own slope-bending and scaling-law-breaking interventions for

As a reminder, here are the two smallest rungs of your Problem 2(a) ladders (the unmodified Problem 2 starter at its default seeds, 42). Each entry is the recipe's final validation loss minus the baseline's at the same depth:

	d4	d6
constant lr	+0.125	+0.187
dropout 0.2	+0.156	+0.161
lr 0.03	+0.023	+0.056

(a) For each non-baseline recipe, check the interval containing $L_{d16}(\text{recipe}) - L_{d16}(\text{baseline})$:

	< 0	0–0.08	0.08–0.18	0.18–0.4	≥ 0.4 or diverges
constant lr	.	.	.	.	.
dropout 0.2	.	.	.	.	.
lr 0.03	.	.	.	.	.

(b) For the standard recipe, $L(d9) - L(d4) = -0.41$. For each intervention, check the interval containing $L(d9) - L(d4)$:

	< -0.25	-0.25 to 0	0 to +0.5	> +0.5
8 epochs over the first 75,000 sequences	.	.	.	.
no QK-norm, lr 0.006	.	.	.	.

Scoring: 3 points per row (five rows), for the interval containing the key runs' difference (bins exhaustive; a boundary value belongs to the lower interval; a non-finite loss counts as the last interval); 1.5 points per row for an adjacent interval.

Mark your answer and write your reasoning on answer sheet page S2.

Q8. One flipped token

Difficulty: Hard

15 points

With full determinism enabled by the patch below, two runs that differ in *exactly one token of one training sequence* (one token of 614.4M, position 100 of the first sequence of step 0's batch changed to token ID 17) are compared step-by-step against each other. We do this across five paired seeds $s \in \{42, 43, 44, 45, 46\}$, setting both the model seed and the data seed to s , and average the measured quantities over those five pairs. We do this twice: once at the standard recipe, and once with only the learning rate raised:

```
--- a/experiments/a1_basics/p4_amplification.py
+++ b/experiments/a1_basics/p4_amplification.py
```

```

@@ -3,8 +3,17 @@ from train import TrainConfig

RUNS = [
-     TrainConfig(deterministic=True),
-     TrainConfig(deterministic=True, perturb_one_token=True),
+     TrainConfig(
+         deterministic=True,
+         model_seed=seed,
+         data_seed=seed,
+         learning_rate=learning_rate,
+         perturb_one_token=(role == "one-token-edit"),
+         run_name_suffix=f"quiz-q6-seed{seed}-{regime}-{role}",
+     )
+     for seed in [42, 43, 44, 45, 46]
+     for regime, learning_rate in [("standard", 0.003), ("lr0.009", 0.009)]
+     for role in ["reference", "one-token-edit"]
]

```

(a) For each pair, take the peak over training steps of the per-step $|\Delta_{\text{train loss}}|$ between the two runs. At $\text{lr } 0.009$, the five-seed average of this peak is:

- $< 2 \times 10^{-2}$
- $2 \times 10^{-2} - 2 \times 10^{-1}$
- $> 2 \times 10^{-1}$

(b) Let σ be the seed-to-seed standard deviation of final validation loss at the standard recipe (the quantity in part (a) of the first question). At the standard recipe, the five-seed average of the *final* $|\Delta_{\text{val}}|$ between paired runs is:

- less than 0.2σ
- between 0.2σ and 5σ
- more than 5σ

Scoring: (a) 7.5 points, (b) 7.5 points; in each part the intervals are exhaustive and non-overlapping (boundaries belong to the left interval), and the correct option is the interval containing the measured five-seed average.

Mark your answer and write your reasoning on answer sheet page [S2](#).

Appendix: scoring code *Read-only derivation from the recorded experiment logs*

The reported outcome is final validation loss from the last evaluation record (the evaluation after the final optimizer step). Train-loss and norm quantities use the per-step W&B records. Every run launched from a question's diff has a `run_name_suffix` of the form `quiz-<question>-<name>`; the program below looks runs up by that name, so running the diffs and then this file (`-fetch ENTITY/PROJECT -prefix quiz-`) reproduces the grading end to end. The published key runs are the same configurations with the prefix `basics-answer-key-v1-` (the internal question ids in run names, such as `q1` or `s2`, predate the current question numbering). The program only reads logs; it does not launch or modify experiments.

A.1 — Q1: what normal training noise looks like

```
def s2_noise():
    """Normal noise: (a) seed SD of the final loss; (b) mean late-training jitter."""
    jitter = statistics.mean(late_jitter(train_losses("s2", f"seed{seed}")) for seed in range(4, 14))
    return {"a": closest(seed_sd(), [0.0003, 0.001, 0.003, 0.01, 0.03]),
            "b": closest(jitter, [0.004, 0.013, 0.04, 0.13])}
```

A.2 — Q2: loss-curve microstructure

```
def q1_jitter():
    """Jitter ratios vs the batch-64 reference -> closest listed ratio."""
    reference = late_jitter(train_losses("q1", "standard-reference"))
    ratio16 = late_jitter(train_losses("q1", "batch-size-16")) / reference
    ratio256 = late_jitter(train_losses("q1", "batch-size-256")) / reference
    options = [0.25, 0.5, 1, 2, 4]
    return {"bs16": closest(ratio16, options), "bs256": closest(ratio256, options)}
```

A.3 — Q3: skipping warmup

```
def s1_no_warmup():
    """Skipping warmup: bin of L(no warmup) - L(standard)."""
    bins = [
        (<0.01", -math.inf, 0.01),
        ("0.01-0.04", 0.01, 0.04),
        ("0.04-0.15", 0.04, 0.15),
        ("0.15-0.5", 0.15, 0.5),
        (">=0.5 or diverges", 0.5, math.inf),
    ]
    return pick(final_val("s1", "no-warmup") - final_val("s1", "standard"), bins)
```

A.4 — Q4: trimming the decay phase

```
def q2_wsd():
    """Q2: smallest decay fraction within 0.02 of the default linear run."""
    linear = final_val("q2", "linear-reference")
    for label, name in [
        ("2%", "wsd0.02"),
        ("5%", "wsd0.05"),
        ("10%", "wsd0.1"),
        ("20%", "wsd0.2"),
        ("50%", "wsd0.5"),
    ]:
        if final_val("q2", name) - linear < 0.02:
            return label
    return "none"
```

A.5 — Q5: peak LR and warmup

```
def q3_warmup_lr():
    """Q3: per (lr, warmup) row, the bin containing the final val loss."""
    bins = [
        ("<2.94", -math.inf, 2.94),
        ("2.94-3.05", 2.94, 3.05),
        ("3.05-3.6", 3.05, 3.6),
        (">=3.6 or NaN", 3.6, math.inf),
    ]
    rows = {
        "lr0.009_wu5%": "lr0.009-warmup0.05",
        "lr0.027_wu5%": "lr0.027-warmup0.05",
        "lr0.027_wu20%": "lr0.027-warmup0.2",
    }
    return {row: pick(final_val("q3", name), bins) for row, name in rows.items()}
```

A.6 — Q6: norm trajectories

```
def q4_norms():
    """Q4(a): final weight norm vs its running peak, per (schedule, wd) cell.
    Q4(b): does the pre-clip grad norm at least double, early -> late?
    """
    peak_bins = [
        ("at its peak (within 3%)", 0.97, 1.0),
        ("3-15% below its peak", 0.85, 0.97),
        (">15% below its peak", 0.0, 0.85),
    ]
    cells = {
        "linear_wd0.1": "linear-wd0.1",
        "linear_wd0": "linear-wd0",
        "const_wd0.1": "constant-wd0.1",
        "const_wd0": "constant-wd0",
    }
    out = {}
    for cell, name in cells.items():
        weights = [value for _step, value in series("q4", name, "weight_norm")]
        out[f"a.{cell}"] = pick(weights[-1] / max(weights), peak_bins)
        grads = [value for _step, value in series("q4", name, "grad_norm")]
        tenth = len(grads) // 10
        early = statistics.median(grads[:tenth])
        late = statistics.median(grads[-tenth:])
        out[f"b.{cell}"] = "checked" if late >= 2 * early else "unchecked"
    return out
```

A.7 — Q7: extrapolating your scaling ladders

```
def q5_ladders():
    """(a) d16 gap to the baseline per P2(a) recipe; (b) L(d9) - L(d4) per intervention."""
    gap_bins = [
        ("<0", -math.inf, 0.0),
        ("0-0.08", 0.0, 0.08),
        ("0.08-0.18", 0.08, 0.18),
        ("0.18-0.4", 0.18, 0.4),
        (">=0.4 or diverges", 0.4, math.inf),
    ]
    depth_bins = [
        ("< -0.25", -math.inf, -0.25),
        (" -0.25 to 0", -0.25, 0.0),
        ("0 to +0.5", 0.0, 0.5),
        ("> +0.5 or diverges", 0.5, math.inf),
    ]
    base = final_val("q5", "d16-base")
    out = {}
    for recipe in ["constant", "dropout0.2", "lr0.03"]:
        out[f"a.{recipe}"] = pick(final_val("q5", f"d16-{recipe}") - base, gap_bins)
    for name in ["epochs8", "noqnorm-lr0.006"]:
        diff = final_val("q5", f"{name}-d9") - final_val("q5", f"{name}-d4")
        out[f"b.{name}"] = pick(diff, depth_bins)
    return out
```

A.8 — Q8: one flipped token

```
def q6_flip():
    """Q6(a): seed-mean peak |dtrain| bin at lr 0.009.
    Q6(b): seed-mean terminal |dval| at the standard recipe, in units of the
    seed-to-seed SD of the final loss (sigma, from the noise question's runs).
    """
    peak_bins = [("<2e-2", 0, 0.02), ("2e-2-2e-1", 0.02, 0.2), (">2e-1", 0.2, math.inf)]
    ratio_bins = [("<0.2 sigma", 0, 0.2), ("0.2-5 sigma", 0.2, 5), (">5 sigma", 5, math.inf)]
    seeds = (42, 43, 44, 45, 46)

    def pair(seed, regime):
        return (f"seed{seed}-{regime}-one-token-edit", f"seed{seed}-{regime}-reference")

    def peak_delta(edited, reference):
        edited_series = series("q6", edited, "train_loss")
        reference_series = series("q6", reference, "train_loss")
        if [step for step, _value in edited_series] != [step for step, _value in reference_series]:
            raise ValueError(f"{edited} / {reference}: unaligned training steps")
        return max(abs(x[1] - y[1]) for x, y in zip(edited_series, reference_series))

    peak = statistics.mean(peak_delta(*pair(seed, "lr0.009")) for seed in seeds)
    terminal = statistics.mean(
        abs(final_val("q6", edited) - final_val("q6", reference))
        for edited, reference in pair(seed, "standard") for seed in seeds
    )
    return {"a": pick(peak, peak_bins), "b.standard": pick(terminal / seed_sd(), ratio_bins)}
```

A.9 — scoring helper functions

These helpers and the functions above constitute the complete, runnable `quiz_scoring.py` answer-key program (Python standard library only, after the W&B cache has been generated).

```
# Final validation loss = the run's last logged val_loss (the evaluation after
# the final optimizer step). Per-step series are the W&B history keys
# train_loss (Q1, Q6) and logging/weight_norm, logging/grad_norm (Q4).
SERIES_KEYS = {"q1": ["train_loss"], "q6": ["train_loss"], "s2": ["train_loss"],
               "q4": ["logging/weight_norm", "logging/grad_norm"]}

def fetch(project, prefix, cache):
    """Download every finished run whose run_name_suffix starts with prefix."""
    import wandb

    rows = []
    for item in wandb.Api().runs(project):
        suffix = item.config.get("run_name_suffix") or ""
        if not suffix.startswith(prefix):
            continue
        series = {}
        for key in SERIES_KEYS.get(suffix[len(prefix):].split("-")[0], []):
            history = item.scan_history(keys=["optimizer_step", key], page_size=5000)
            series[key.split("/")[-1]] = sorted(
                [row["optimizer_step"], row[key]] for row in history
                if row.get(key) is not None)
        rows.append({"run_name_suffix": suffix, "state": item.state,
                    "created_at": str(item.created_at),
                    "val_loss": item.summary.get("val_loss"), "series": series})
    Path(cache).write_text(json.dumps(rows))

def load_runs(cache, prefix):
    """Latest run per suffix, keyed by the suffix with the prefix removed."""
    latest = {}
    for item in json.loads(Path(cache).read_text()):
        suffix = item.get("run_name_suffix") or ""
        if not suffix.startswith(prefix):
            continue
        name = suffix[len(prefix):]
        current = latest.get(name)
        if current is None or item.get("created_at", "") > current.get("created_at", ""):
            latest[name] = item
    return latest

RUNS = {}

def run(question, name):
    suffix = f"{question}-{name}"
    item = RUNS[suffix]
    if item["state"] != "finished":
        raise ValueError(f"{suffix}: state is {item['state']}, cannot score")
    if item.get("val_loss") is None:
        raise ValueError(f"{suffix}: missing final validation loss")
    return item

def final_val(question, name):
    """The final validation loss from the last evaluation record."""
    return run(question, name)["val_loss"]
```

```

def series(question, name, field):
    """A cached per-step series such as train_loss or logging/grad_norm."""
    return run(question, name)["series"][field]

def train_losses(question, name):
    return [value for _step, value in series(question, name, "train_loss")]

def pick(value, bins):
    """The label of the interval containing value; bins = [(label, lo, hi)]."""
    for label, lo, hi in bins:
        if lo <= value <= hi:
            return label
    if math.isnan(value) and any(word in bins[-1][0] for word in ("NaN", "diverges")):
        return bins[-1][0]
    raise ValueError(f"{value}: no printed answer interval contains this value")

def closest(value, options):
    """The listed option closest to value in ratio (smallest |log(value/option)|)."""
    return min(options, key=lambda option: abs(math.log(value / option)))

def late_jitter(losses, tail=2000, w=51):
    """SD of (per-step train loss - centered 51-step mean) over the last 2000 steps."""
    tail_losses = losses[-tail:]
    mean = [sum(tail_losses[i - w // 2 : i + w // 2 + 1]) / w
            for i in range(w // 2, len(tail_losses) - w // 2)]
    return statistics.pstdev(tail_losses[w // 2 + i] - m for i, m in enumerate(mean))

def seed_sd():
    """Sample SD of the final validation loss over the noise question's ten seeds."""
    return statistics.stdev(final_val("s2", f"seed{seed}") for seed in range(4, 14))

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--prefix", default=KEY_PREFIX)
    parser.add_argument("--cache", default=None)
    parser.add_argument("--fetch", metavar="ENTITY/PROJECT", default=None)
    args = parser.parse_args()
    cache = args.cache or (KEY_CACHE if args.prefix == KEY_PREFIX else "quiz_runs.json")
    if args.fetch:
        fetch(args.fetch, args.prefix, cache)
    RUNS.update(load_runs(cache, args.prefix))
    key = {
        "Q1 noise (s2)": s2_noise(),
        "Q2 jitter ratios (q1)": q1_jitter(),
        "Q3 skipping warmup (s1)": s1_no_warmup(),
        "Q4 decay fraction (q2)": q2_wsd(),
        "Q5 lr x warmup (q3)": q3_warmup_lr(),
        "Q6 norms (q4)": q4_norms(),
        "Q7 scaling ladders (q5)": q5_ladders(),
        "Q8 one flipped token (q6)": q6_flip(),
    }
    print(json.dumps(key, indent=2))

```

This page is intentionally left blank.

Answer sheet. Check one box per row (Q6(b): every run that applies) and write 2–3 lines of reasoning that cite your assignment's experiments. Only these pages are graded.

Q1. What normal training noise looks like

10 points

	0.0003	0.001	0.003	0.01	0.03		0.004	0.013	0.04	0.13
(a)	☐	☐	☐	☐	☐	(b)	☐	☐	☐	☐

Reasoning.

Q2. Loss-curve microstructure

10 points

	$\times 1/4$	$\times 1/2$	$\times 1$	$\times 2$	$\times 4$
batch size 16	☐	☐	☐	☐	☐
batch size 256	☐	☐	☐	☐	☐

Reasoning.

Q3. Skipping warmup

10 points

	$\Delta < 0.01$	$0.01 \leq \Delta < 0.04$	$0.04 \leq \Delta < 0.15$	$0.15 \leq \Delta < 0.5$	$\Delta \geq 0.5$ or diverges
Δ	☐	☐	☐	☐	☐

Reasoning.

Q4. Trimming the decay phase

10 points

	2%	5%	10%	20%	50%	none of these
smallest X	☐	☐	☐	☐	☐	☐

Reasoning.

Q5. Peak learning rate and warmup

15 points

lr	warmup	< 2.94	2.94–3.05	3.05–3.6	≥ 3.6 or NaN
0.009	5%	☐	☐	☐	☐
0.027	5%	☐	☐	☐	☐
0.027	20%	☐	☐	☐	☐

Reasoning.

Q6. Norm trajectories under schedule and weight decay

10 points

schedule	wd	(a) at its peak (within 3%)	3–15% below peak	> 15% below peak	(b) grad norm doubles
linear	0.1	☐	☐	☐	☐
linear	0.0	☐	☐	☐	☐
constant	0.1	☐	☐	☐	☐
constant	0.0	☐	☐	☐	☐

Reasoning.

Q7. Extrapolating your scaling ladders

15 points

(a) L_{d16} gap to baseline	< 0	0–0.08	0.08–0.18	0.18–0.4	≥ 0.4 or diverges
constant lr	☐	☐	☐	☐	☐
dropout 0.2	☐	☐	☐	☐	☐
lr 0.03	☐	☐	☐	☐	☐
(b) $L(d9) - L(d4)$	< -0.25	-0.25 to 0	0 to +0.5	> +0.5	
8 epochs over the first 75,000 sequences	☐	☐	☐	☐	
no QK-norm, lr 0.006	☐	☐	☐	☐	

Reasoning.

Q8. One flipped token

15 points

	< 2×10^{-2}	2×10^{-2} – 2×10^{-1}	> 2×10^{-1}		less than 0.2σ	0.2σ – 5σ	more than 5σ
(a)	☐	☐	☐	(b)	☐	☐	☐

Reasoning.